

Neural LSH Vector Search in Python

Τίτλος: Αναζήτηση Διανυσμάτων με Neural LSH (Python)

Περιγραφή: Η εργασία υλοποιεί τον αλγόριθμο Neural LSH για προσεγγιστική αναζήτηση γειτόνων (Approximate Nearest Neighbors - ANN) σε διανύσματα υψηλής διάστασης, χρησιμοποιώντας PyTorch, NumPy, KaHIP και τα παραδοτέα της 1ης εργασίας για την αποδοτική κατασκευή k-NN γράφων. Υποστηρίζονται τα ίδια datasets με την 1η εργασία (MNIST, SIFT) και εκτελείται ακριβής baseline (εξαντλητική αναζήτηση) για υπολογισμό μετρικών AF, Recall@N, QPS και χρόνων.

Στοιχεία Φοιτητών:

1ο μέλος: Ιωάννης Νικολόπουλος - sdi2100292

2ο μέλος: Ιωάννης Πετράκης - sdi1900155

Κατάλογος αρχείων & ρόλοι (source tree)

Παρακάτω περιγράφονται τα βασικά Python αρχεία/μονάδες της 2ης εργασίας:

Αρχείο	Περιγραφή
nlsh_build.py	Κύριο πρόγραμμα κατασκευής ευρετηρίου Neural LSH. Φορτώνει το dataset, κατασκευάζει k-NN γράφο (μέσω C++ approx_knn), μετατρέπει σε weighted graph, τρέχει KaHIP για partitioning και εκπαιδεύει MLP ταξινομητή. Αποθηκεύει το index (μοντέλο + inverted file) σε αρχείο .pt.
nlsh_search.py	Κύριο πρόγραμμα αναζήτησης. Φορτώνει το αποθηκευμένο index, τα δεδομένα και τα queries, εκτελεί Neural LSH (top-T bins, συλλογή υποψηφίων, exact search στους υποψηφίους) και γράφει το αρχείο εξόδου με μετρικές.
arguments.py	Συγκεντρωτικός ορισμός/ανάλυση παραμέτρων γραμμής εντολών για nlsh_build.py και nlsh_search.py (build_args, search_args). Ορίζει όλα τα flags, τα default και την περιγραφή τους.
dataset_parser.py	Φόρτωση datasets. Υλοποιεί parser για MNIST (idx3-ubyte) και SIFT (.fvecs) και εκθέτει συναρτήσεις load_dataset, load_mnist_images, load_sift_fvecs, καθώς και default ακτίνες R για τα δύο datasets.

graph_utils.py	Λειτουργίες γύρω από τον k-NN γράφο και τη διαμέριση: Enum KnnMethods (lsh, hypercube, ivfflat, ivfpq) και συνάρτηση external_knn που καλεί το εκτελέσιμο C++ knn/bin/approx_knn για να κατασκευάσει k-NN γράφο. Περιέχει brute_force_knn για πλήρως εξαντλητικό k-NN, μετατροπή k-NN σε προς KaHIP γράφο (knn_to_weighted_undirected_edges, edges_to_csr), και KaHIPParams & build_partitions για κλήση kahip.kaffra και παραγωγή labels 0..m-1.
models.py	Ορισμός MLP ταξινομητή (MLPClassifier) σε PyTorch, μαζί με MLPConfig και βοηθητικές train_mlp_classifier (training loop) και predict_top_T_bins για επιλογή των T bins με τη μεγαλύτερη softmax πιθανότητα.
index_io.py	Αποθήκευση/φόρτωση ευρετηρίου Neural LSH. Η save_index(...) γράφει σε .pt όλα τα απαραίτητα στοιχεία (dataset_type, shape, k για k-NN, παραμέτρους KaHIP, MLP config, weights μοντέλου, labels, inverted index). Η load_index(...) επαναδημιουργεί το MLP, φορτώνει labels & inverted index και επιστρέφει ένα αντικείμενο Index.
search_utils.py	Ακριβείς (εξαντλητικές) συναρτήσεις αναζήτησης πάνω σε NumPy arrays: exact_knn και exact_range για ακτίνα R. Χρησιμοποιούνται από nlsh_search.py για baseline/μετρικές.
precompute_knn.py	Μικρό βοηθητικό script για πλήρη brute-force υπολογισμό k-NN γράφου (με brute_force_knn) και αποθήκευση/φόρτωση σε .npy. Χρήσιμο για μικρά subsets ή για debugging της προσέγγισης.
requirements.txt	Λίστα Python εξαρτήσεων (NumPy, PyTorch, KaHIP, κλπ.). Εγκατάσταση με pip install -r requirements.txt.
knn/bin/approx_knn	Εκτελέσιμο C++ (παραδοτέο 1ης εργασίας) τροποποιημένο ώστε να δουλεύει με πολλά threads) που υλοποιεί LSH / Hypercube / IVFFlat / IVFPQ και χρησιμοποιείται εδώ σαν backend για την κατασκευή του k-NN γράφου (βλ. graph_utils.external_knn). Πρέπει να έχει ήδη μεταγλωττιστεί από την 1η εργασία.

(Ο φάκελος μπορεί επίσης να περιέχει τα C++ αρχεία της 1ης εργασίας για αναφορά ή για μεταγλώττιση του approx_knn.)

Προαπαιτούμενα

Λειτουργικό: Linux (όπως ζητείται στην εκφώνηση).

Python: 3.10+.

Εργαλεία μεταγλώττισης: g++, make, libtbb-dev για τη μεταγλώττιση του C++ approx_knn (1η εργασία).

Βιβλιοθήκες Python (μέσω requirements.txt):

- numpy
- torch
- kahip

Εγκατάσταση εξαρτήσεων

1. Δημιουργία virtual environment (προαιρετικό αλλά προτεινόμενο)

```
python3 -m venv venv  
source venv/bin/activate
```

2. Εγκατάσταση Python βιβλιοθηκών

```
pip install --upgrade pip  
pip install -r requirements.txt
```

3. Μεταγλώττιση C++ approx_knn (από την 1η εργασία)

Από τον φάκελο της 1ης εργασίας (ή από υποφάκελο knn/ της παρούσας):

```
make
```

Το εκτελέσιμο πλέον υπάρχει στη διαδρομή:

```
knn/bin/approx_knn
```

(Αυτό το path χρησιμοποιείται από τη graph_utils.external_knn για να καλέσει το πρόγραμμα του k-NN.)

Μορφή datasets

Χρησιμοποιούνται ακριβώς τα ίδια δυαδικά αρχεία με την 1η εργασία:

MNIST: idx3-ubyte (big-endian), 784-διάστατα διανύσματα uint8. Διαβάζονται σε float32 πίνακα σχήματος [n, 784].

SIFT: .fvecs (little-endian), 128-διάστατα διανύσματα float32. Διαβάζονται σε float32 πίνακα σχήματος [n, 128].

Ο φάκελος data/ (ή κάποιος άλλος) θα πρέπει να περιέχει τα αντίστοιχα αρχεία:

- mnist-data.dat, mnist-query.dat
- sift_base.fvecs, sift_query.fvecs

Οδηγίες χρήσης (CLI)

Η υλοποίηση χωρίζεται σε δύο κύρια σενάρια:

1. Κατασκευή ευρετηρίου (build) με nlsh_build.py
2. Αναζήτηση (search) με nlsh_search.py

1. Κατασκευή Ευρετηρίου - nlsh_build.py

Βασικό usage:

```
python nlsh_build.py \
-d <DATA_FILE> \
-i <INDEX_PATH> \
-type {mnist|sift} \
[--knn K] \
[--knn_method {lsh|hypercube|ivfflat|ivfpq}] \
[-m M] \
[--imbalance EPS] \
[--kahip_mode {0|1|2}] \
[--layers L] \
[--nodes H] \
[--epochs E] \
[--batch_size B] \
[--lr LR] \
[--seed S]
```

Υποχρεωτικά flags

-d <DATA_FILE>: dataset base (π.χ. data/mnist-data.dat, data/sift_base.fvecs).

-i <INDEX_PATH>: διαδρομή όπου θα αποθηκευτεί το ευρετήριο Neural LSH (π.χ. indexes/mnist_nlsh.pt).

-type {mnist|sift}: τύπος δεδομένων.

K-NN γράφος / backend

--knn <int>: αριθμός γειτόνων k για k-NN γράφο. Default: 10.

--knn_method {lsh|hypercube|ivfflat|ivfrq}: ποια μέθοδος της 1ης εργασίας θα χρησιμοποιηθεί από το approx_knn για να κατασκευάσει τον k-NN γράφο. Default: ivfflat.

Εσωτερικά καλείται:

knn/bin/approx_knn -d <DATA_FILE> -type <mnist|sift> -<method> -N <knn>

KaHIP / διαμέριση χώρου

-m <int>: αριθμός partitions/bins m. Default: 100.

--imbalance <float>: ε, επιτρεπόμενη ανισορροπία KaHIP. Default: 0.03.

--kahip_mode {0|1|2}: λειτουργία KaHIP (0=FAST, 1=ECO, 2=STRONG). Default: 2 (STRONG).

--seed <int>: κοινό seed για KaHIP, MLP κλπ. Default: 1.

MLP ταξινομητής (Neural LSH)

--layers <int>: πλήθος στρωμάτων του MLP (συμπεριλαμβανομένου του output). Default: 3.

--nodes <int>: αριθμός hidden units ανά κρυφό στρώμα. Default: 64.

--epochs <int>: αριθμός εποχών εκπαίδευσης. Default: 10.

--batch_size <int>: μέγεθος batch κατά την εκπαίδευση. Default: 128.

--lr <float>: learning rate. Default: 0.001.

2. Αναζήτηση - nlsh_search.py

Βασικό usage:

```
python nlsh_search.py \  
-d <DATA_FILE> \  
-q <QUERY_FILE> \  
-i <INDEX_PATH> \  
-o <OUTPUT_FILE> \  
-type {mnist|sift} \  
[-N N] \  
[-R R] \  
[-T T] \  
[-range {true|false}]
```

Υποχρεωτικά flags

-d <DATA_FILE>: base dataset (ίδιο αρχείο με αυτό που χρησιμοποιήθηκε στο build).

-q <QUERY_FILE>: set ερωτημάτων (π.χ. data/mnist-query.dat, data/sift_query.fvecs).

-i <INDEX_PATH>: index που παρήχθη από nlsh_build.py (π.χ. indexes/mnist_nlsh.pt).

-o <OUTPUT_FILE>: αρχείο εξόδου (π.χ. out/mnist_nlsh.txt).

-type {mnist|sift}: τύπος δεδομένων.

Neural LSH / αναζήτηση

-N <int>: πλήθος πλησιέστερων γειτόνων που επιστρέφονται. Default: 1.

-T <int>: πλήθος bins T (top-T) που θα ελεγχθούν (multi-probe). Default: 5.

Range search

-R <float>: ακτίνα για range search. Default:

MNIST -> 2000.0

SIFT -> 2800.0

(αν δεν δοθεί, επιλέγεται αυτόματα ανάλογα με το -type).

-range {true|false}: αν θα εκτελεστεί και range search. Default: true.

Παραδείγματα εκτέλεσης

1. MNIST - Κατασκευή ευρετηρίου

```
python nlsh_build.py \  
-d data/mnist-data.dat \  
-i indexes/mnist_nlsh.pt \  
-type mnist \  
--knn 15 \  
--knn_method ivfflat \  
-m 100 \  
--epochs 10 \  
--layers 3 \  
--nodes 64
```

2. MNIST - Αναζήτηση

```
python nlsh_search.py \  
-d data/mnist-data.dat \  
-q data/mnist-query.dat \  
-i indexes/mnist_nlsh.pt \  
-o out/mnist_nlsh.txt \  
-type mnist \  
-N 10 \  
-T 5 \
```

```
-R 2000 \  
-range true
```

3. SIFT - Κατασκευή ευρετηρίου

```
python nlsh_build.py \  
-d data/sift_base.fvecs \  
-i indexes/sift_nlsh.pt \  
-type sift \  
--knn 20 \  
--knn_method ivfflat \  
-m 100 \  
--epochs 10 \  
--layers 3 \  
--nodes 128
```

4. SIFT - Αναζήτηση

```
python nlsh_search.py \  
-d data/sift_base.fvecs \  
-q data/sift_query.fvecs \  
-i indexes/sift_nlsh.pt \  
-o out/sift_nlsh.txt \  
-type sift \  
-N 10 \  
-T 5 \  
-R 2800 \  
-range true
```

Μορφή εξόδου & μετρικές

Το αρχείο εξόδου (-o) έχει μορφή παρόμοια με της 1ης εργασίας, με method name Neural LSH. Ενδεικτικά:

Neural LSH

Query: 0

Nearest neighbor-1: 12345

distanceApproximate: 1172.43

distanceTrue: 1172.43

Nearest neighbor-2: 6789

distanceApproximate: 1300.12

distanceTrue: 1299.50

...

R-near neighbors: 596

Average AF: 1.008000

Recall@N: 0.980000

QPS: 55.000000

tApproximateAverage: 18.200000

tTrueAverage: 120.000000

Query: i : id ερωτήματος (γραμμή i στο query set).

Για κάθε Nearest neighbor-k τυπώνεται:

distanceApproximate: απόσταση από τον προσεγγιστικό αλγόριθμο (Neural LSH).

distanceTrue: απόσταση από την ακριβή αναζήτηση (exhaustive baseline).

R-near neighbors: πλήθος σημείων εντός ακτίνας R (στην παρούσα υλοποίηση καταγράφεται ο αριθμός τους).

Στο τέλος του αρχείου τυπώνονται:

Average AF: μέσο Approximation Factor για το 1-NN.

Recall@N: κλάσμα queries όπου ο ακριβής 1-NN ανήκει στους N προσεγγιστικούς γείτονες.

QPS: queries per second, βάσει του προσεγγιστικού χρόνου.

tApproximateAverage: μέσος χρόνος (ms) Neural LSH ανά query.

tTrueAverage: μέσος χρόνος (ms) πλήρους ακριβούς αναζήτησης ανά query.

Σημείωση: Αν χρειαστεί πλήρης λίστα R-near neighbors (όπως στην 1η εργασία), μπορεί εύκολα να ενεργοποιηθεί η σχετική εκτύπωση μέσα στο nlsh_search.py.

Περιγραφή υλοποίησης Neural LSH (σύνοψη)

Σε πολύ υψηλό επίπεδο, η υλοποίηση ακολουθεί τα 4 βήματα της εκφώνησης:

Κατασκευή k-NN γράφου: Χρήση ενός από τους αλγόριθμους της 1ης εργασίας (LSH / Hypercube / IVFFlat / IVFPQ) μέσω του C++ approx_knn για παραγωγή k-NN γράφου $G = (P, E)$.

Μετατροπή σε weighted, μη κατευθυνόμενο γράφο: Τα directed k-NN edges μετατρέπονται σε undirected edges με βάρη 2 για αμοιβαίους γείτονες, 1 για μονόπλευρους, και στη συνέχεια μετατρέπονται σε CSR μορφή (vwgt, xadj, adjncy, adjcwt).

Ισοκατανεμημένη διαμέριση με KaHIP: Κλήση `kahip.kaffra` για διαμέριση του γράφου σε m μέρη με ισορροπημένα μεγέθη, παράγοντας $\text{labels } \pi(p) \in \{0, \dots, m-1\}$ για κάθε σημείο.

Εκπαίδευση MLP (το οποίο προβλέπει ένα σημείο σε ποια διαμέριση του χώρου ανήκει και άρα την ετικέτα του) ταξινομητή & inverted index: Εκπαίδευση MLP $M: \mathbb{R}^d \rightarrow \{0, \dots, m-1\}$ που μαθαίνει να προβλέπει ένα σημείο σε ποια διαμέριση του χώρου ανήκει και άρα την ετικέτα του κάθε σημείου. Στη συνέχεια κατασκευάζεται inverted index $r \rightarrow \{p_i : \pi(p_i)=r\}$.

Κατά την αναζήτηση: Υπολογίζεται $M(q)$ και επιλέγονται τα T bins με τη μεγαλύτερη softmax πιθανότητα, συλλέγονται όλοι οι υποψήφιοι από τα αντίστοιχα bins, εκτελείται ακριβές k -NN πάνω στους υποψηφίους και (προαιρετικά) εκτελείται και exact range search για ακτίνα R .

Πειραματική μελέτη

Η αναλυτική τεκμηρίωση της πειραματικής συγκριτικής μελέτης (επιλογή υπερπαραμέτρων, σύγκριση Neural LSH με LSH/Hypercube/IVFFlat/IVFPQ ως προς AF, Recall@N, QPS, μνήμη, κλπ.) παρέχεται σε ξεχωριστό παραδοτέο (π.χ. `report.md` ή PDF), όπως ζητείται στην εκφώνηση.